

Modern Cryptography

Digital Signatures

Plan

1. 전자서명의 개념
2. 보안 모델과 위조 분류
3. Plain RSA 서명과 취약점
4. 해시 기반 RSA 서명
5. DL 기반 서명
6. 무작위성과 해싱

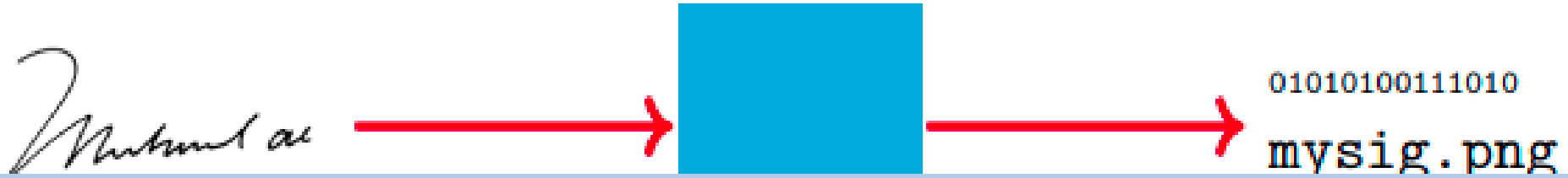
전자서명의 개념

손글씨 서명의 한계



- 물리적 서명: 문서에 대한 서명자 동의를 증명하는 전통적 수단
- 디지털화의 문제: 스캔 이미지는 단순 복사/붙여넣기로 쉽게 위조 및 도용 가능
- 전자서명 필수 요건: 안전한 서명은 '서명자'뿐 아니라 '메시지 내용'에도 의존해야 함

손글씨 서명의 한계



**동일한 서명자라도 대상 메시지가 다르면 서명값 자체가
완전히 달라지도록 설계해야 함**

- 물
- 디지털화의 문제: 스캔 이미지는 단순 복사/붙여넣기로 쉽게 위조 및 도용 가능
- 전자서명 필수 요건: 안전한 서명은 '서명자'뿐 아니라 '메시지 내용'에도 의존해야 함

전자서명의 작동 원리

- 기반 기술: 공개키와 비밀키로 이루어진 비대칭 키 쌍 구조
- 서명 생성: 본인만 소유한 비밀키를 이용해 해당 메시지에 고유 서명 부여
- 서명 검증: 공개된 검증키를 적용해 해당 서명이 유효한지 확인

전자서명의 작동 원리

1. $(vk, sk) \leftarrow K$ (키 쌍 생성 후 분배)
2. 메시지 M 준비
3. $\sigma \leftarrow S_{sk}(M)$ (서명 생성)
4. $V_{vk}(M, \sigma) = 1?$ (서명 검증 후 수락/거부)

전자서명 vs MAC

- MAC (대칭키 기반): 송수신자가 동일한 단일 키를 공유
- MAC의 약점: 검증 서버 해킹 및 키 유출 시, 공격자가 완벽하게 서명 위조 가능
- 디지털 서명 (비대칭키 기반): 검증키(vk)와 서명키(sk)가 분리됨
- 디지털 서명의 방어력: 서버가 해킹되어 검증키(vk)가 유출되어도, 서명 생성 자체는 불가능해 위조 원천 차단

전자서명 vs MAC

	MAC	전자서명
키 구조	대칭키 1개	키 쌍 (sk/vk)
검증 주체	키 보유자만	누구나
서버 침해 시	위조 가능	위조 불가

Syntax

전자서명 스킴 $\mathcal{DS} = (\mathcal{K}, \mathcal{S}, \mathcal{V})$ 는 다음 세 가지 다항시간 알고리즘으로 구성된다

1. 키 생성 $\mathcal{K}: (vk, sk) \leftarrow \mathcal{K}$
 - 검증키(vk)와 서명키(sk) 쌍을 생성한다
2. 서명 생성 $\mathcal{S}: \sigma \leftarrow \mathcal{S}_{sk}(M)$
 - 서명키 sk 와 메시지 M 을 입력받아 서명 σ 를 출력한다
3. 서명 검증 $\mathcal{V}: d \leftarrow \mathcal{V}_{vk}(M, \sigma)$
 - 검증키 vk , 메시지 M , 서명 σ 를 입력받아 유효하면 1, 아니면 0을 출력한다

정확성 요건

$$\Pr[\mathcal{V}_{vk}(M, \mathcal{S}_{sk}(M)) = 1] = 1 \text{ for all } (vk, sk)$$

- 기본 전제: 합법적 사용자가 정상적인 비밀키로 생성한 서명은 검증 과정을 반드시 항상 통과해야 함
- 작동 일관성: 키 생성이나 서명 과정에 무작위 요소가 섞여 매번 값이 달라져도, 최종 검증은 100% 수락(1)
- 암호학적 정의: 정확성 요건을 완벽한 확률 수식($Pr = 1$)으로 엄밀하게 정의

보안 모델과 위조 분류

UF-CMA

Game $\text{UFCMA}_{\mathcal{DS}}$

procedure Initialize

$(vk, sk) \xleftarrow{\$} \mathcal{K}; S \leftarrow \emptyset$

return vk

procedure Finalize(M, σ)

$d \leftarrow \mathcal{V}_{vk}(M, \sigma)$

return $(d = 1 \wedge M \notin S)$

procedure Sign(M)

$\sigma \xleftarrow{\$} \mathcal{S}_{sk}(M)$

$S \leftarrow S \cup \{M\}$

return σ

- UF-CMA 정의: Chosen Message Attack 하에서의 Existential Unforgeability 모델

UF-CMA

- 공격자 조건: 공개 검증키(vk)를 보유하고, 원하는 임의의 메시지에 대해 마음껏 합법적 서명을 요청/획득 가능
- 승리 조건: 서명 요청을 한 적 없는 새로운 메시지에 대해 유효한 서명을 제출
- 방어 목표: 공격자에게 가장 유리한 극한의 환경에서도 새로운 서명 위조가 불가능함을 증명

각 프로시저의 의미

- 목적: 서명 보안성 증명을 위한 공격자-방어자 간의 수학적 게임 모델링
- Initialize 단계: 방어자 측 키 쌍 생성 및 공격자에게 공개키(vk) 부여
- Sign 단계: 공격자가 원하는 메시지 오라클 요청 → 획득 후 시스템에 요청 목록(S) 기록
- Finalize 단계: 공격자가 위조 서명 제출 → 시스템이 최종 유효성 판별 및 승패 결정

승리 조건

- 게임 승리를 위한 2가지 동시 충족 조건
- 조건 1 (Correct): 제출한 서명이 시스템 검증 알고리즘을 정상 통과
 $(\mathcal{V}_{vk}(M, \sigma) = 1)$
- 조건 2 (New): 제출한 메시지가 이전 서명 요청 목록(S)에 없는 완전한 새 메시지 ($M \notin S$)

승리 조건

- 게임 승리를 위한 2가지 동시 충족 조건

- 조
(12
- 조
메

Sign은 서명자를, Finalize는 검증자를 표현한다.
공격자가 서명자가 이전에 서명하지 않은 메시지를
검증자가 수락하도록 만들 수 없음을 의미한다.

새

서명 위조의 분류 체계

공격 달성 목표 난이도에 따른 3단계 분류 체계

위조 유형	정의	공격 난이도
Existential	공격자가 (M, σ) 쌍을 자유롭게 선택	쉬움
Selective	공격자가 오라클 호출 전에 M 을 미리 선택	중간
Universal	공격자가 임의의 메시지에 서명 가능	어려움

서명 위조의 분류 체계

공격 달성 목표 난이도에 따른 3단계 분류 체계

위조 유형	정의	공격 난이도
-------	----	--------

Existential	UF-CMA의 철학: 가장 난이도가 낮은 Existential조차 달성하지 못하게 철저히 차단	
-------------	---	--

Selective	공격자가 오라클 호출 전에 M 을 미리 선택	중간
-----------	----------------------------	----

Universal	공격자가 임의의 메시지에 서명 가능	어려움
-----------	---------------------	-----

Plain RSA 서명과 취약점

Plain RSA 서명 스킴

- 모델 정의: 전통적인 RSA 암호 생성기를 아무런 변형 없이 서명 용도로 직접 활용
- 서명 도출: 서명자 비밀키(d) 지수를 메시지에 적용하여 모듈러 거듭제곱 수행 (RSA 역함수)
- 서명 검증: 공개키(e) 지수로 서명값을 거듭제곱하여 원래 메시지 값과 직접 비교 대조

Plain RSA 서명 스킴

알고리즘	입력	출력	연산
$S(N,d)(y)$	메시지 $y \in \mathbb{Z}_N^*$	서명 x	$x \leftarrow y^d \pmod{N}$
$V(N,e)(y,x)$	메시지 y , 서명 x	$d \in 0,1$	$x^e \pmod{N} \stackrel{?}{=} y$

정확성 증명

오일러의 정리 기반의 대수학적 특성 활용

- 오일러 정리: $\gcd(a, N) = 1$ 이면, $a^{\phi(N)} \equiv 1 \pmod{N}$
- 지수 관계: $ed = 1 + k\phi(N)$
- 정확성 증명: $x^e \pmod{N} = (y^d)^e \pmod{N} = y^{ed} \pmod{\phi(N)} = y^1 = y$

예시1

- 키 생성: $p = 11, q = 13 \Rightarrow N = 143, \phi(N) = 120$
- $e = 7$ ($\gcd(7, 120) = 1$)
- $d \equiv 7^{-1} \pmod{120} \Rightarrow 7 \times 103 = 721 = 6 \times 120 + 1 \Rightarrow d = 103$
- 서명: $y = 2 \Rightarrow x = 2^{103} \pmod{143} = 6$
- 검증: $6^7 \pmod{143} = 279936 \pmod{143} = 2 = y$ (검증 성공)

취약점1: Trivial Forgery

Adversary $A((N, e))$

return (1, 1)

- 공격 원리: "1의 거듭제곱은 항상 1이다"라는 진리 악용
- 공격 수법: 메시지 1과 서명값 1의 조합인 (1, 1) 쌍을 검증자에게 단순 제출
- 성립 사유: 비밀키(d)의 정체와 무관하게 $1^d \pmod{N} = 1$ 조건이 무조건 성립
- 결과: 단 한 번의 서명 오라클 호출 없이도 완벽한 위조 게임 승리 ($Adv = 1$)

취약점2: Multiplicative Homomorphism

Adversary $A((N, e))$:

$y_1, y_2 \leftarrow \mathbb{Z}_N^* \setminus \{1\}$

$x_1 \leftarrow \text{Sign}(y_1)$

$x_2 \leftarrow \text{Sign}(y_2)$

return $(y_1 * y_2 \bmod N, x_1 * x_2 \bmod N)$

- 공격 원리: RSA 알고리즘의 대수적 특성인 곱셈 준동형성 악용
- 공격 절차: 임의의 두 메시지에 대해 합법적인 서명값 (x_1, x_2) 각각 획득

취약점2: Multiplicative Homomorphism

- 위조 기법: 획득한 두 서명값을 곱하기만 하면, 원본 메시지들의 곱(y_1y_2)에 대한 완벽한 합법 서명값으로 둔갑
- 취약점 원인: 복잡해야 할 수학적 대수 구조가 시스템 겉면에 그대로 노출됨

성공 이유 수식: $(y_1y_2)^d \pmod N = y_1^d \cdot y_2^d \pmod N = x_1 \cdot x_2 \pmod N$

준동형성 함수식: $f^{-1}(y_1y_2) = f^{-1}(y_1) \cdot f^{-1}(y_2)$

취약점3: Inversion Attack

Adversary $A((N, e))$:

$x \leftarrow \$ Z^*_N$

$y \leftarrow x^e \bmod N$

return (y, x)

- 공격 원리: 정상적인 "메시지 $\rightarrow$ 서명" 생성 순서를 역전시킴
- 공격 절차: 무작위 숫자(x)를 먼저 "서명값"이라 우기고, 공개키(e)로 역연산하여 "결과값(y)" 강제 생성

취약점3: Inversion Attack

- 위조 기법: 결과값(y)을 원본 메시지로, 초기 무작위(x)를 서명값으로 짝지어 제출
- 결과: 내용 없는 쓰레기 데이터라도 서명 오라클 도움 없이 완벽하게 유효한 서명 쌍 창출

성공 이유 수식: $y^d \pmod N = (x^e)^d \pmod N = x^{ed} \pmod{\phi(N)} = x$

일방향성의 한계

Game $\text{OW}_{\mathcal{K}_{\text{rsa}}}$

procedure Initialize

$(N, p, q, e, d) \xleftarrow{\$} \mathcal{K}_{\text{rsa}}$
 $x \xleftarrow{\$} \mathbb{Z}_N^*; y \leftarrow x^e \bmod N$
return N, e, y

procedure Finalize(x')

return $(x = x')$

- 랜덤으로 주어지는 출력값의 역추적이 어렵다는 것만 보장할 뿐임

일방향성의 한계

Game $\text{UFCMA}_{\mathcal{DS}}$

procedure Initialize

$(vk, sk) \xleftarrow{\$} \mathcal{K}; S \leftarrow \emptyset$
return vk

procedure Finalize(M, σ)

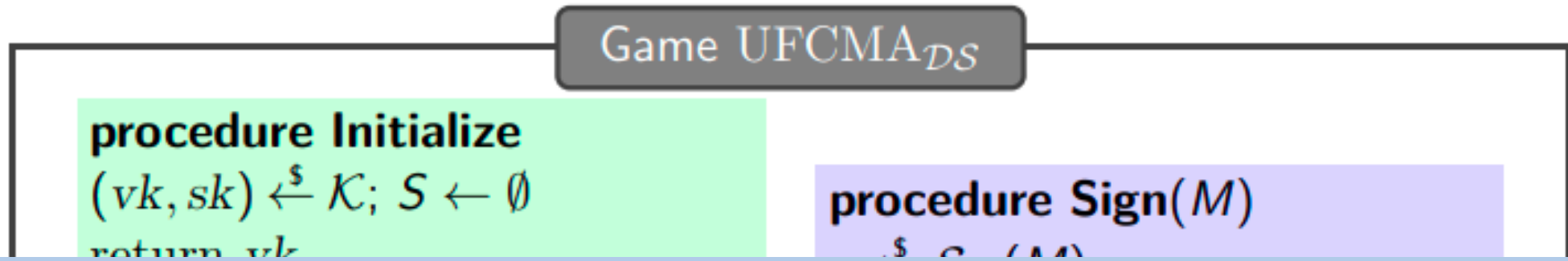
$d \leftarrow \mathcal{V}_{vk}(M, \sigma)$
return $(d = 1 \wedge M \notin S)$

procedure Sign(M)

$\sigma \xleftarrow{\$} \mathcal{S}_{sk}(M)$
 $S \leftarrow S \cup \{M\}$
return σ

- UF-CMA 게임의 본질: 서명 공격자는 무작위 값이 아닌, 스스로 조작하기 쉬운 '특수 형태'의 메시지를 골라 공격

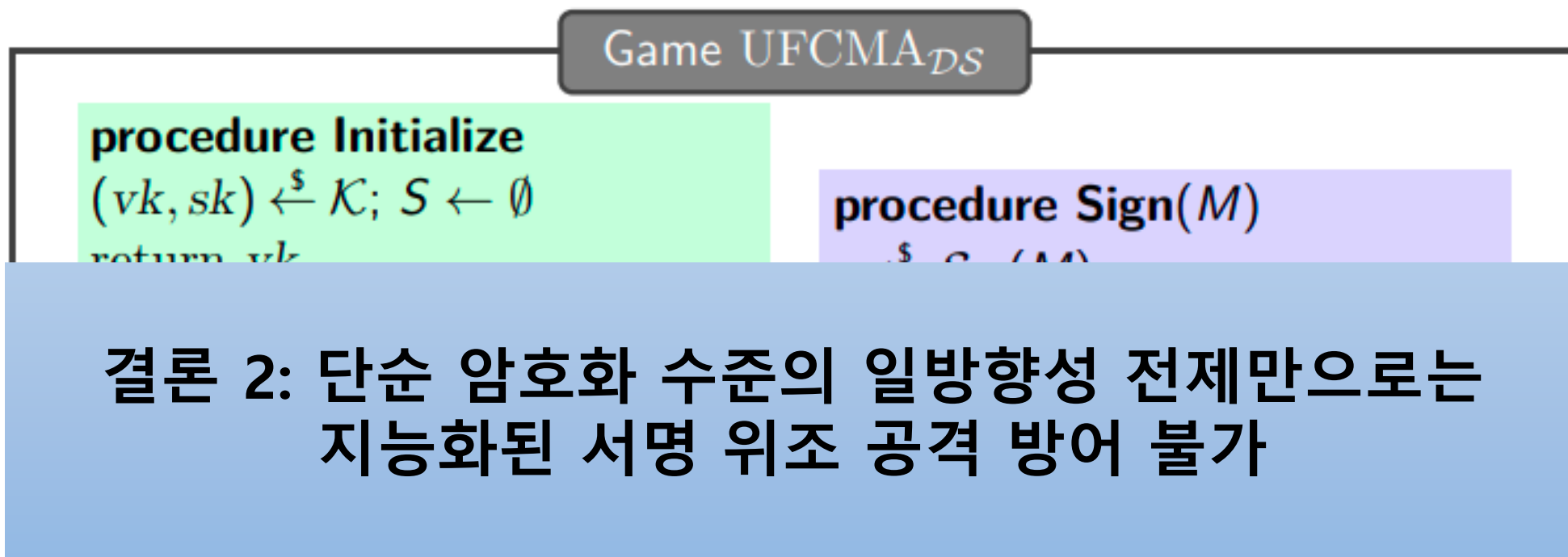
일방향성의 한계



**결론 1: 방어자가 주는 문제를 푸는 것(OW)과
해커가 스스로 문제를 만드는 것(UF-CMA)은 차원이 다름**

- UF-CMA 게임의 본질: 서명 공격자는 무작위 값이 아닌, 스스로 조작하기 쉬운 '특수 형태'의 메시지를 골라 공격

일방향성의 한계



- UF-CMA 게임의 본질: 서명 공격자는 무작위 값이 아닌, 스스로 조작하기 쉬운 '특수 형태'의 메시지를 골라 공격

해시 기반 RSA 서명

PKCS#1 서명 구조

Plain RSA의 문제를 완화하기 위해 메시지를 특정 형태로 인코딩한다

- 거대 정수 공간 $\mathbb{Z}_N^*$ 원소가 아무 형태나 띠지 못하고 특정 비트 구조 패턴을 따르도록 강제
- 메시지 해시(SHA-1 등) → 고정 패딩 바이트(00 01 FF...) 결합 → 최종 RSA 연산 적용

목표: 입력값 형태를 까다롭게 제한하여 악의적인 산술 조작 사전 차단

PKCS#1 서명 구조

- $|N| = 1024$ bits ($n = 128$ bytes)
- 패딩 함수식: $H_{PKCS}(M) = 00||01||FF|| \dots ||FF||h(M)$ (해시 20 bytes)
- 서명 생성식: $S_{N,d}(M) = H_{PKCS}(M)^d \pmod{N}$

PKCS#1의 한계

- 구조적 결함: 128바이트 중 무려 108바이트가 고정 패턴이고 오직 20바이트 (해시)만 변동
- 데이터 불균형: 인위적으로 편향된 비트 구조 탓에 전체 모듈러 공간 $\mathbb{Z}_N^*$ 에서 고른 균등 분포 달성 실패
- 증명 한계: '균등 랜덤 입력'을 절대 전제로 하는 RSA 일방향성 모델로의 논리적 환원 불가

Full Domain Hash (FDH)

PKCS#1의 수학적 증명 불가 굴레를 끊어낸 해시 서명 스킴 (1996)

- 핵심 아이디어: 메시지를 특정 부분만 해싱하지 않고 $\mathbb{Z}_N^*$ 의 거대한 전체 공간 크기로 꽉 채워 확장
- 작동 방식: 공간을 가득 채운 거대 해시값 위에 직접 RSA 역함수를 적용
- 결과: 출력값이 전체 공간에 고르게 퍼지는 '균등 분포' 달성 → 엄밀한 수학적 증명의 굳건한 토대 마련

Full Domain Hash (FDH)

알고리즘	연산
$\mathcal{K}$	$(N, p, q, e, d) \stackrel{?}{\leftarrow} \mathcal{K}_{rsa}, vk = (N, e), sk = (N, d)$
$\mathcal{S}_{(N,d)}(M)$	$y \leftarrow H(M); x \leftarrow y^d \pmod{N}; \text{return } x$
$\mathcal{V}_{(N,e)}(M, x)$	$y \leftarrow H(M); \text{return } (x^e \pmod{N} \stackrel{?}{=} y)$

충돌 저항성의 필요성

- 밑바탕이 되는 해시 함수(H)에 충돌이 발생할 경우 전체 방어 시스템 즉시 붕괴
- 공격 과정: 내용은 다르나 해시값이 동일한 쌍(M_1, M_2) 탐색 $\rightarrow M_1$ 의 합법 서명을 M_2 서명으로 무단 복제 제출

충돌 저항성의 필요성

Adversary $A((N, e))$:

$(M_1, M_2) \leftarrow \$ C$ // 해시 충돌 알고리즘

$x_1 \leftarrow \text{Sign}(M_1)$ // M_1 의 서명 획득

return (M_2, x_1) // M_2 서명으로 위조 제출

- 성공 이유:

$$H(M_1) = H(M_2) \implies \mathcal{S}(M_1) = H(M_1)^d \pmod{N} = H(M_2)^d \pmod{N} = \mathcal{S}(M_2)$$

충돌 저항성의 필요성

Adversary $A((N, e))$:

충돌 저항성(Collision Resistance)은 서명 안전성을 지키는 필수적인 전제 조건

`return (M2, x1) // M2가 M1과 같은 해시 값을 가지도록 선택`

- 성공 이유:

$$H(M_1) = H(M_2) \Rightarrow \mathcal{S}(M_1) = H(M_1)^d \pmod{N} = H(M_2)^d \pmod{N} = \mathcal{S}(M_2)$$

충돌 저항성의 필요성

Adversary $A((N, e))$:

**한계: 필수조건이긴 하나,
완전한 UF-CMA 방어를 위한 충분조건은 아님**

return (M2, x1) // M2 성공으로 뒤로 세력

- 성공 이유:

$$H(M_1) = H(M_2) \Rightarrow \mathcal{S}(M_1) = H(M_1)^d \pmod{N} = H(M_2)^d \pmod{N} = \mathcal{S}(M_2)$$

Z_N vs Z_N^*

- SHAKE256 같은 가변 출력 길이 함수(XOF)를 동원해 이론상 요구되는 긴 비트 수로 해시 확장 구현
- 이론상 가역원 그룹($\mathbb{Z}_N^*$) 출력이 필요하나, 실제 구현은 전체 정수 범위($\mathbb{Z}_N$)로 도출됨
- 출력값이 비가역원(p 나 q 의 배수)으로 맞아떨어질 확률은 무시할 수 있을 만큼 작음 ($\approx \frac{2}{\sqrt{N}}$)

Random Oracle 모델에서의 FDH 안전성

- H 를 Random Oracle로 모델링 한 때 RSA FDH이 UF-CMA 안전성의 RSA의 **수학적 손실을 없애줄 촘촘한 서명 스킴(PSS) 연구의 기폭제 역할**
- q_H 는 공격자 A 의 해시 쿼리 횟수
- 문제점: 쿼리 공격 횟수에 정비례하여 방어 시스템의 이론적 안전성이 깎여나가는 손실 발생

RSA PSS

- FDH의 수학적 한계를 극복하고 등장한 규격이자, 현존 최고 수준의 확률론적 RSA 서명 스킴
- 차별점: 서명할 때마다 매번 다르게 생성되는 무작위 '솔트(Salt)' 값 투입

RSA PSS

Alg $\mathcal{S}_{(N,d)}(M)$

$r \xleftarrow{\$} \{0, 1\}^\ell$

$w \leftarrow \mathbf{H}_1(M \parallel r)$

$r^* \leftarrow \mathbf{H}_2(w) \oplus r$

$y \leftarrow 0 \parallel w \parallel r^* \parallel \mathbf{H}_3(w)$

$x \leftarrow y^d \bmod N$

Return x

- 3단계 해시 구역(H_1, H_2, H_3)을 통해 원본 메시지와 솔트 값을 정교하게 얹고 마스킹 압축
- 방어 효과: 해커가 수집된 서명값을 역으로 분석하여 원본 메시지의 패턴이나 구조를 유추하는 관찰 시도 원천 차단

검증 알고리즘

Alg $\mathcal{V}_{(N,e)}(M, x)$

$y \leftarrow x^e \bmod N$

$b \parallel w \parallel r^* \parallel P \leftarrow y$

$r \leftarrow r^* \oplus \mathbf{H}_2(w)$

If $(\mathbf{H}_3(w) \neq P)$ then return 0

If $(b = 1)$ then return 0

If $(\mathbf{H}_1(M \parallel r) \neq w)$ then return 0

Return 1

1. 역산: 서명 수신자가 검증키(e)로 RSA 정방향 계산을 돌려 겹겹이 쌓인 인코딩 데이터 파싱 해체
2. 복원: 해시 함수의 자기 역원성(XOR) 성질을 교묘히 이용해, 마스킹된 겹질을 벗기고 순수 솔트값 색출

검증 알고리즘

Alg $\mathcal{V}_{(N,e)}(M, x)$

$y \leftarrow x^e \bmod N$

$b \parallel w \parallel r^* \parallel P \leftarrow y$

$r \leftarrow r^* \oplus \mathbf{H}_2(w)$

If $(\mathbf{H}_3(w) \neq P)$ then return 0

If $(b = 1)$ then return 0

If $(\mathbf{H}_1(M \parallel r) \neq w)$ then return 0

Return 1

- 다중 검사: 패리티 비트($H_3(w)$) 이상 무 확인 $\rightarrow$ 최상위 오류 방지 0비트 세팅 점검
- 핵심 승인: 복원된 솔트와 제시된 메시지를 다시 뭉쳐 해싱한 값이 내부 w 구역과 100% 일치할 때만 서명 최종 수락

PSS의 우수성

- 증명 안전성을 깎아먹던 해시 쿼리 인수(q_H) 제거 → 상대적으로 짧은 RSA 키로도 최상급 방어력 구현
- 동일한 원본 메시지라도 매번 전혀 다른 서명값을 뱉어내 관찰 및 패턴 유추 공격 무력화
- 이론적 완벽함과 실용적 강점을 앞세워 현행 주요 국제 보안 프로토콜의 최고 규격으로 쓰임

표준화 목록

- RSA PKCS#1 v2.1, v2.2 / RFC 8017
- IEEE P1363a
- ANSI X9.31
- RFC 3447
- ISO/IEC 9796-2

DL 기반 서명

Schnorr 서명 스킴

- 무겁고 느린 소수 곱셈(RSA)을 탈피, 빠르고 가벼운 타원 곡선 등 '순환 군' 체계 도입
- 공개키 $X = g^x$ 에서 아무리 쳐다봐도 원래 비밀키 스칼라 x 를 역추적하기 불가능하다는 점 이용 (DLP)

Schnorr 서명 스킴

Alg $\mathcal{K}$

$x \xleftarrow{s} \mathbb{Z}_m$

$X \leftarrow g^x$

Return (X, x)

Alg $\mathcal{S}_x^H(M)$

$r \xleftarrow{s} \mathbb{Z}_m ; R \leftarrow g^r$

$c \leftarrow \mathbf{H}(R \| M)$

$s \leftarrow (r + cx) \bmod m$

Return (R, s)

1. 무작위 주사위(Nonce, r)를 던져 초기값 발급 $\rightarrow$ 공개 커밋먼트 생성
2. 메시지와 커밋먼트를 섞은 해시 챌린지 생성 $\rightarrow$ 최종 서명 응답(s) 신속 산출

Schnorr 검증 알고리즘

Alg $\mathcal{V}_X^H(M, (R, s))$

If $(R \notin G)$ then return 0

$c \leftarrow \mathbf{H}(R \| M)$

If $(g^s = RX^c)$ then return 1

Else return 0

- 수신자가 건네받은 데이터(R, M)를 기반으로 서명자의 해시 챌린지 값(c)을 똑같이 재생산
- 서명자가 제출한 응답 승수(g^s) 결과를 좌변에 올리고 비교
- 우변에 커밋먼트(R)와 챌린지 씌운 공개키(X^c) 조합을 올려, 양쪽 무게가 단 1비트의 오차 없이 같은지 대조

Schnorr 검증 알고리즘

Alg $\mathcal{V}_X^H(M, (R, s))$

If $(R \notin G)$ then return 0

$c \leftarrow H(R \| M)$

If $(s \neq 0)$

Else

$$\text{정확성 증명 유도식: } g^s = g^{r+cx} = g^r \cdot (g^x)^c = R \cdot X^c$$

• 수

(c)

을 똑같이 재생산

• 서명자가 제출한 응답 승수(g^s) 결과를 좌변에 올리고 비교

• 우변에 커밋먼트(R)와 챌린지 씌운 공개키(X^c) 조합을 올려, 양쪽 무게가 단 1비트의 오차 없이 같은지 대조

Schnorr 검증 알고리즘

Alg $\mathcal{V}_X^H(M, (R, s))$

If $(R \notin G)$ then return 0

$c \leftarrow H(R \| M)$

If $(s \neq 0)$

Else

양변이 완벽히 일치하면,
진짜 비밀키 소유자만이 풀었음이 입증됨

• 수

을 똑같이 재생산

(c)

• 서명자가 제출한 응답 승수(g^s) 결과를 좌변에 올리고 비교

• 우변에 커밋먼트(R)와 챌린지 씌운 공개키(X^c) 조합을 올려, 양쪽 무게가 단 1비트의 오차 없이 같은지 대조

EdDSA

- 이론적 Schnorr 알고리즘을 실용성과 고속 처리 능력으로 끌어올린 타원 곡선 완성작
- 랜덤 발급된 키를 해시로 확장 후, 구조적으로 가장 튼튼하고 안전한 위수를 가지도록 조각하는 작업 수행
- 외부 난수 생성기의 돌발 고장에 대비하여, 무작위 Nonce 사용 안함
- 본인만의 비밀키와 메시지 조각을 결합하여 흔들리지 않는 고유 난수 발생 → 최고 등급 TLS 규격 채택

DSA / ECDSA

- DSA 기반 구축: 1991년 NIST 제안, 기존 El Gamal 기법을 고도화하여 정부 차원 빠르게 표준화
- ECDSA로의 진화: 무거운 유한체 기반 DSA 코어를 덜어내고, 작고 빠른 타원 곡선 환경 위로 포팅 이식
- 블록체인 생태계 점령: 가벼운 용량과 빠른 처리 속도로 비트코인, 이더리움 지갑 및 자산 보호용 핵심 기술로 등극

Schnorr vs ECDSA

특성	Schnorr/EdDSA	ECDSA
서명 크기	2×32 bytes (Ed25519)	2×32 bytes
키 집약	가능	불가
Tight security reduction	존재	존재하지 않음
결정론적 서명	기본 제공	RFC 6979로 별도 구현 필요
특허 이슈	없음	현재 만료

무작위성과 해싱

서명에서의 무작위성

- 서명의 본질: 매번 형태를 감춰야 하는 '암호화'와 달리, 본인 인증 목적의 서명은 굳이 무작위 패턴이 강제되지 않음
- 다만, 알고리즘 태생상 DSA/Schnorr는 서명 시 반드시 일회용 난수(Nonce)를 뽑아야 동작함

예시2

- 불량 난수 생성기 탓에 서로 다른 두 문서에 동일한 Nonce가 재사용되는 순간, 간단한 뺄셈만으로 서명자 비밀키 1초 만에 유출
- Nonce r 이 같을 경우 서명식 2개 연립:
 - $s_1 = r + c_1x \pmod{m}$, $s_2 = r + c_2x \pmod{m}$
 - $s_1 - s_2 = (c_1 - c_2)x \pmod{m} \Rightarrow x = \frac{s_1 - s_2}{c_1 - c_2} \pmod{m}$ (비밀키 x 직접 노출)

해싱의 필수성1

해시 과정 없이 짧은 문자열 메시지 본문에 직접 거듭제곱 서명 시,
Plain RSA와 동일한 공격이 작동한다.

- Trivial Forgery : 입력창에 그저 숫자 '1'만 입력해도 무조건 '1'로 출력되어 패스 됨
- Homomorphism: 짧은 두 메시지의 서명값을 악의적으로 곱해도 모듈러 범위를 넘지 않아, 곱셈 공격 가능

해싱의 필수성2

강력한 암호학적 해시 대신 일반 통신망 단순 오류 체크섬인 'CRC16'을 서명 패딩용으로 남용

- 16비트(65,536개)라는 짧은 길이로 무차별 대입 시 즉각적인 해시 충돌 유발
- CRC는 단순한 선형 결합 구조이기에 $CRC = 0$ 이 나오는 짝을 골라내 곱셈 조작하면 방어벽 뚫림

해싱의 필수성2

강력한 암호화 알고리즘을 사용하더라도 해시 함수를 사용하면 데이터의 무결성을 검증할 수 있다. 서명

패딩

**결론: 서명에서 검사 역할을 하는 함수는 반드시
암호학적 해시 함수의 성질을 가져야 한다.**

• 16비트

유발

- CRC는 단순한 선형 결합 구조이기에 $CRC = 0$ 이 나오는 짝을 골라내 곱셈 조작하면 방어벽 뚫림